

Programmazione funzionale (OCaml)

<https://taroccoesbrocco.github.io/progfunz.html>

Esonero 2 — 9 giugno 2026

Esercizio 1 (35 punti)

Si consideri la struttura dati per gli alberi binari con nodi di tipo `'a` definita a lezione come

```
type 'a tree = Empty | Tr of 'a * 'a tree * 'a tree
```

1. (5 punti) Si definisca una funzione `add.tree : char tree -> char tree -> char tree list -> char tree list` tale che `add.tree left right lst` aggiunga l'albero binario `Tr('x',left,right)` in testa alla lista `lst` di alberi binari.

Se `left` è un albero binario, qual è il tipo di `add.tree left`?

2. (10 punti) Utilizzando la funzione `add.tree` del punto precedente, si definisca una funzione `add.left.to.rights : char tree -> char tree list -> char tree list -> char tree list` tale che `add.left.to.rights left rights lst` costruisca l'albero binario `Tr('x',left,r)` per ogni albero binario `r` nella lista `rights` e l'aggiunga in testa alla lista `lst` di alberi binari.

Per esempio, `add.left.to.rights (Tr('l',Empty,Empty)) [Tr('r',Empty,Empty); Tr('R',Empty,Empty)] [Tr('a',Empty,Empty); Empty] =`

```
[Tr ('x', Tr ('l', Empty, Empty), Tr ('r', Empty, Empty));  
 Tr ('x', Tr ('l', Empty, Empty), Tr ('R', Empty, Empty));  
 Tr ('a', Empty, Empty); Empty].
```

Suggerimento: Si utilizzi la funzione `List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`.

3. (10 punti) Utilizzando la funzione `add.left.to.rights` del punto precedente, si definisca una funzione `add.lefts.to.rights : char tree list -> char tree list -> char tree list -> char tree list` tale che `add.lefts.to.rights lefts rights lst` costruisca l'albero binario `Tr('x',l,r)` per ogni albero binario `l` nella lista `lefts` e ogni albero binario `r` nella lista `rights`, e l'aggiunga in testa alla lista `lst` di alberi binari.

Per esempio, `add.lefts.to.rights [Tr('l',Empty,Empty); Tr('L',Empty,Empty)] [Tr('r',Empty,Empty); Tr('R',Empty,Empty)] [Tr('a',Empty,Empty); Empty] =`

```
[Tr ('x', Tr ('l', Empty, Empty), Tr ('r', Empty, Empty));  
 Tr ('x', Tr ('l', Empty, Empty), Tr ('R', Empty, Empty));  
 Tr ('x', Tr ('L', Empty, Empty), Tr ('r', Empty, Empty));  
 Tr ('x', Tr ('L', Empty, Empty), Tr ('R', Empty, Empty));  
 Tr ('a', Empty, Empty); Empty].
```

Suggerimento: Si utilizzi la funzione `List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`.

4. (10 punti) Un albero binario è *bilanciato* se la seguente proprietà è vera per ogni nodo: il numero di nodi del suo sottoalbero sinistro differisce al più di 1 rispetto al numero di nodi del suo sottoalbero destro.

Si definisca una funzione `bal.trees : int -> char tree list` tale che `bal.trees n` restituisca la lista di tutti gli alberi binari bilanciati di `n` nodi, dove ciascun nodo è etichettato dal carattere `'x'`.

Per esempio, `bal.trees 4 =`

```
[Tr ('x', Tr ('x', Empty, Empty), Tr ('x', Tr ('x', Empty, Empty), Empty));  
 Tr ('x', Tr ('x', Empty, Empty), Tr ('x', Empty, Tr ('x', Empty, Empty)));  
 Tr ('x', Tr ('x', Tr ('x', Empty, Empty), Empty), Tr ('x', Empty, Empty));  
 Tr ('x', Tr ('x', Empty, Tr ('x', Empty, Empty)), Tr ('x', Empty, Empty))].
```

Suggerimento: Si utilizzi la funzione `add.lefts.to.rights` del punto precedente e si distinguano i casi in cui `n` è uguale a 0 oppure è dispari oppure è pari (diverso da 0).